

```

# =====
# BLOB Principe - SAUT 3.6 : EXPOSANTS CRITIQUES & TRANSITION DE PHASE
# =====
#
# Objectif : tester l intuition "Blob = transition de phase du second ordre"
# en mesurant les exposants critiques pres de tau_c ~ 0.40.
#
# Methode :
# 1. Scan tau dans [0.30, 0.55] (12 valeurs)
# 2. A chaque tau : 3 seeds, N fixe ~ 200, 100k equilibration + 100k mesure
# 3. Mesurer parametre d ordre PHI, susceptibilite chi, chaleur specifique C
# 4. Ajuster les exposants critiques alpha, beta, gamma
# 5. Comparer aux classes d universalite connues
#
# Sorties :
# - Profils PHI(tau), chi(tau), C(tau)
# - Position pic susceptibilite -> tau_c affine
# - Exposants beta (parametre d ordre) et gamma (susceptibilite)
# - Match avec Ising 3D / XY 3D / Heisenberg 3D / Champ moyen / Percolation
# =====

import sys
import os

def auto_check():
    print("=" * 70)
    print("  AUTO-TEST CORRUPTION CARACTERES")
    print("=" * 70)
    try:
        if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv
            source_file = sys.argv[0]
        else:
            source_file = __file__ if '__file__' in globals() else None
        if source_file and os.path.exists(source_file):
            with open(source_file, 'r', encoding='utf-8') as f:
                content = f.read()
            dangerous = {chr(8216): "U+2018", chr(8217): "U+2019",
                chr(8220): "U+201C", chr(8221): "U+201D",
                chr(8211): "U+2013", chr(8212): "U+2014"}
            found = [(name, content.count(c)) for c, name in dangerous.items() if
            if found:
                print("ERREUR caracteres corrompus :", found)
                return False
            print("OK : pas de corruption")

```

```

        else:
            print("Note : auto-test partiel")
    except Exception as e:
        print("Note : " + str(e))
    print("=" * 70)
    return True

if not auto_check():
    sys.exit(1)

import numpy as np
import time
import json
import gc
from collections import Counter

KAPPA_0 = 8.0
BETA_VIAB = 0.30 # parametre du modele (PAS 1 exposant critique)
EPS_DECAL = 0.05

OUTPUT_DIR = '/content/blob_saut36_results'
os.makedirs(OUTPUT_DIR, exist_ok=True)

# =====
# Classe BlobUniverse a N FIXE (pas de mitose pendant la mesure)
# =====
class BlobUniverseFixedN:
    """Version a N fixe : on initialise un univers de N simplexes,
    pas de mitose, juste dynamique de viabilite avec tau parametrable."""

    def __init__(self, N, tau, seed=0):
        self.rng = np.random.default_rng(seed)
        self.N = N
        self.tau = tau
        self.seed = seed
        # Positions aleatoires pour le pool Rips
        self.positions = [self.rng.random(3) for _ in range(N)]
        self.adj = [set() for _ in range(N)]
        self.deg = np.zeros(N, dtype=np.int32)
        self.edge_list = []
        self.n_tri = 0
        self.n_tetra = 0
        self.S_curr = 0.0
        # Initialisation: ajouter aleatoirement des aretes jusqu a kappa_0*N/2
        self._initialize_edges()

```

```

def _initialize_edges(self):
    E_target = int(KAPPA_0 * self.N / 2)
    attempts = 0
    max_attempts = E_target * 100
    while len(self.edge_list) < E_target and attempts < max_attempts:
        u = int(self.rng.integers(self.N))
        v = int(self.rng.integers(self.N))
        attempts += 1
        if u == v or v in self.adj[u]:
            continue
        u, v = min(u, v), max(u, v)
        dt_add = len(self.adj[u] & self.adj[v])
        self.adj[u].add(v)
        self.adj[v].add(u)
        self.deg[u] += 1
        self.deg[v] += 1
        self.edge_list.append((u, v))
        self.n_tri += dt_add
        # tetras (dlentete a chaque ajout)
        common = self.adj[u] & self.adj[v]
        if len(common) >= 2:
            cl = list(common)
            for i, w in enumerate(cl):
                for x in cl[i+1:]:
                    if x in self.adj[w]:
                        self.n_tetra += 1
    self.S_curr = self._compute_S()

def _compute_D(self):
    mu = float(self.deg.mean())
    if mu == 0:
        return 0.0
    return float(self.deg.std()) / mu

def _compute_S(self):
    return np.log(1 + self.n_tri) - BETA_VIAB * self._compute_D()

def _count_tetra_through_edge(self, u, v):
    common = self.adj[u] & self.adj[v]
    if len(common) < 2:
        return 0
    cl = list(common)
    count = 0
    for i, w in enumerate(cl):
        adj_w = self.adj[w]
        for x in cl[i+1:]:

```

```

        if x in adj_w:
            count += 1
    return count

def _list_tetrahedra(self):
    result = []
    seen = set()
    for u, v in self.edge_list:
        common = self.adj[u] & self.adj[v]
        if len(common) < 2:
            continue
        cl = sorted(common)
        for i, w in enumerate(cl):
            for x in cl[i+1:]:
                if x in self.adj[w]:
                    key = tuple(sorted([u, v, w, x]))
                    if key not in seen:
                        seen.add(key)
                        result.append(key)
    return result

def compute_PHI(self):
    if self.n_tetra == 0:
        return 0.0
    tetras = self._list_tetrahedra()
    if not tetras:
        return 0.0
    edge_to_tetras = {}
    for tet in tetras:
        for i in range(4):
            for j in range(i+1, 4):
                e = (tet[i], tet[j])
                edge_to_tetras.setdefault(e, []).append(tet)
    neighbors_count = {tet: set() for tet in tetras}
    for e, tlist in edge_to_tetras.items():
        if len(tlist) > 1:
            for i, t1 in enumerate(tlist):
                for t2 in tlist[i+1:]:
                    neighbors_count[t1].add(t2)
                    neighbors_count[t2].add(t1)
    total = sum(len(v) for v in neighbors_count.values())
    return float(total) / len(tetras)

def step_swap(self):
    """Pas Blob : swap (retirer arete + ajouter arete) avec acceptance Metrop
    if not self.edge_list:
        return False

```

```

idx = int(self.rng.integers(len(self.edge_list)))
u_rm, v_rm = self.edge_list[idx]
u_add = int(self.rng.integers(self.N))
v_add = int(self.rng.integers(self.N))
if u_add == v_add or v_add in self.adj[u_add]:
    return False
u_add, v_add = min(u_add, v_add), max(u_add, v_add)
if (u_add, v_add) == (u_rm, v_rm):
    return False
dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
dtetra_rm = self._count_tetra_through_edge(u_rm, v_rm)
self.adj[u_rm].discard(v_rm)
self.adj[v_rm].discard(u_rm)
self.deg[u_rm] -= 1
self.deg[v_rm] -= 1
dt_add = len(self.adj[u_add] & self.adj[v_add])
self.adj[u_add].add(v_add)
self.adj[v_add].add(u_add)
self.deg[u_add] += 1
self.deg[v_add] += 1
dtetra_add = self._count_tetra_through_edge(u_add, v_add)
new_n_tri = self.n_tri - dt_rm + dt_add
if new_n_tri < 0:
    # rollback
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False
new_S = np.log(1 + new_n_tri) - BETA_VIAB * self._compute_D()
delta_S = new_S - self.S_curr
log_alpha = self.N * delta_S / self.tau
accepted = (log_alpha >= 0) or (self.rng.random() < np.exp(max(log_alpha,
if accepted:
    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.n_tetra += (dtetra_add - dtetra_rm)
    self.S_curr = new_S
    return True
else:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1

```

```

        self.deg[v_add] -= 1
        self.adj[u_rm].add(v_rm)
        self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1
        self.deg[v_rm] += 1
        return False

# =====
# Mesure d un point (tau, seed)
# =====
def measure_point(N, tau, seed, n_equil, n_measure, n_samples=200):
    """Equilibre puis mesure observables. Retourne moyenne + variance."""
    sim = BlobUniverseFixedN(N, tau, seed)

    # Equilibration
    for _ in range(n_equil):
        sim.step_swap()

    # Mesure : echantillonner regulierement
    sample_interval = max(1, n_measure // n_samples)
    PHI_samples = []
    D_samples = []
    S_samples = []
    n_tri_samples = []
    n_tetra_samples = []

    for step in range(n_measure):
        sim.step_swap()
        if step % sample_interval == 0:
            PHI_samples.append(sim.compute_PHI())
            D_samples.append(sim._compute_D())
            S_samples.append(sim.S_curr)
            n_tri_samples.append(sim.n_tri)
            n_tetra_samples.append(sim.n_tetra)

    PHI_arr = np.array(PHI_samples)
    D_arr = np.array(D_samples)
    S_arr = np.array(S_samples)

    return {
        'tau': tau,
        'seed': seed,
        'N': N,
        'n_samples': len(PHI_samples),
        'PHI_mean': float(PHI_arr.mean()),
        'PHI_var': float(PHI_arr.var()),

```

```

        'D_mean': float(D_arr.mean()),
        'D_var': float(D_arr.var()),
        'S_mean': float(S_arr.mean()),
        'S_var': float(S_arr.var()),
        # Susceptibilite chi = N * Var(D) (analogue M = D, chi = N * <(M-<M>)^2>)
        'chi': float(N * D_arr.var()),
        # Chaleur specifique-like : C = N * Var(S)
        'C': float(N * S_arr.var()),
        'n_tri_mean': float(np.mean(n_tri_samples)),
        'n_tetra_mean': float(np.mean(n_tetra_samples)),
    }

# =====
# Analyse des exposants critiques
# =====
def fit_exponent(tau_vals, obs_vals, tau_c, side='right'):
    """Fit log-log de obs ~ |tau - tau_c|^exposant.
    side='right' : tau > tau_c (parametre d ordre)
    side='left' : tau < tau_c
    side='both' : symetrique (susceptibilite, chaleur spec)
    Retourne exposant et r2.
    """
    tau_arr = np.array(tau_vals)
    obs_arr = np.array(obs_vals)

    # Filtre selon le cote
    if side == 'right':
        mask = tau_arr > tau_c
    elif side == 'left':
        mask = tau_arr < tau_c
    else:
        mask = tau_arr != tau_c

    if mask.sum() < 3:
        return None, None

    tau_diff = np.abs(tau_arr[mask] - tau_c)
    obs_filt = obs_arr[mask]

    # Filtre valeurs positives (log requis)
    valid = (tau_diff > 1e-9) & (obs_filt > 1e-9)
    if valid.sum() < 3:
        return None, None

    log_tau = np.log(tau_diff[valid])
    log_obs = np.log(obs_filt[valid])

```

```

# Regression lineaire :  $\log(\text{obs}) = \text{exposant} * \log(|\text{tau}-\text{tau}_c|) + \text{cst}$ 
coeffs = np.polyfit(log_tau, log_obs, 1)
slope = float(coeffs[0])

# R2
pred = np.polyval(coeffs, log_tau)
ss_res = np.sum((log_obs - pred)**2)
ss_tot = np.sum((log_obs - log_obs.mean())**2)
r2 = 1 - ss_res/ss_tot if ss_tot > 0 else 0

return slope, float(r2)

# =====
# Classes d'universalite connues
# =====
UNIVERSALITY = {
    'Champ moyen': {'beta': 0.500, 'gamma': 1.000, 'nu': 0.500},
    'Ising 3D': {'beta': 0.326, 'gamma': 1.237, 'nu': 0.630},
    'XY 3D': {'beta': 0.348, 'gamma': 1.316, 'nu': 0.671},
    'Heisenberg 3D': {'beta': 0.367, 'gamma': 1.388, 'nu': 0.707},
    'Percolation 3D': {'beta': 0.418, 'gamma': 1.793, 'nu': 0.876},
    'Ising 2D': {'beta': 0.125, 'gamma': 1.750, 'nu': 1.000},
}

# =====
# MAIN
# =====
print('')
print('=' * 75)
print(' BLOB PRINCIPLE - SAUT 3.6 - EXPOSANTS CRITIQUES')
print('=' * 75)

# Configuration
N_FIXED = int(os.environ.get('BLOB_N', 200))
TAU_MIN = float(os.environ.get('TAU_MIN', 0.30))
TAU_MAX = float(os.environ.get('TAU_MAX', 0.55))
N_TAU = int(os.environ.get('N_TAU', 12))
N_SEEDS = int(os.environ.get('N_SEEDS', 3))
N_EQUIL = int(os.environ.get('N_EQUIL', 100000))
N_MEASURE = int(os.environ.get('N_MEASURE', 100000))

tau_grid = np.linspace(TAU_MIN, TAU_MAX, N_TAU)

print(f' N fixe : {N_FIXED}')

```



```

print(f' Scan tau : {TAU_MIN} a {TAU_MAX} ({N_TAU} valeurs)')
print(f' Seeds par tau : {N_SEEDS}')
print(f' Equilibration : {N_EQUIL} pas')
print(f' Mesure : {N_MEASURE} pas')
print(f' Duree estimee : ~{N_TAU * N_SEEDS * 2} min')
print('=' * 75)

t_start = time.time()
all_measurements = []

for tau_idx, tau in enumerate(tau_grid):
    print('')
    print(f' --- tau = {tau:.4f} ({tau_idx+1}/{N_TAU}) ---', flush=True)
    for seed in range(N_SEEDS):
        t0 = time.time()
        m = measure_point(N_FIXED, tau, seed, N_EQUIL, N_MEASURE)
        all_measurements.append(m)
        elapsed = time.time() - t0
        print(f' seed {seed} : PHI={m["PHI_mean"]:.3f} chi={m["chi"]:.4f} C={m["C"]:.4f} D={m["D_mean"]:.4f} n_tetra={m["n_tetra_mean"]:.4f}')
        gc.collect()

total_time = time.time() - t_start
print('')
print('=' * 75)
print(f' Toutes mesures terminees en {total_time/60:.1f} min')
print('=' * 75)

# =====
# AGREGATION : moyenne sur seeds par tau
# =====
agg = {}
for m in all_measurements:
    tau = m['tau']
    if tau not in agg:
        agg[tau] = {'PHI': [], 'chi': [], 'C': [], 'D_mean': [], 'n_tetra': []}
    agg[tau]['PHI'].append(m['PHI_mean'])
    agg[tau]['chi'].append(m['chi'])
    agg[tau]['C'].append(m['C'])
    agg[tau]['D_mean'].append(m['D_mean'])
    agg[tau]['n_tetra'].append(m['n_tetra_mean'])

tau_sorted = sorted(agg.keys())
PHI_means = [np.mean(agg[t]['PHI']) for t in tau_sorted]
chi_means = [np.mean(agg[t]['chi']) for t in tau_sorted]
C_means = [np.mean(agg[t]['C']) for t in tau_sorted]
D_means = [np.mean(agg[t]['D_mean']) for t in tau_sorted]

```

```

print('')
print(' --- Profils par tau ---')
print(f' {"tau":>6s} {"PHI":>8s} {"chi":>10s} {"C":>10s} {"D":>8s} {"n_tetr
for i, t in enumerate(tau_sorted):
    print(f' {t:>6.3f} {PHI_means[i]:>8.3f} {chi_means[i]:>10.4f} {C_means[i]

# =====
# Identifier tau_c affine = position du pic de chi
# =====
chi_arr = np.array(chi_means)
peak_idx = int(np.argmax(chi_arr))
tau_c_measured = tau_sorted[peak_idx]
chi_peak = chi_arr[peak_idx]
print('')
print(f' Pic chi a tau_c = {tau_c_measured:.4f} (chi = {chi_peak:.4f})')

# Pic chaleur specifique
C_arr = np.array(C_means)
peak_idx_C = int(np.argmax(C_arr))
tau_c_C = tau_sorted[peak_idx_C]
print(f' Pic C a tau_c = {tau_c_C:.4f} (C = {C_arr[peak_idx_C]:.4f})')

# =====
# Fit exposants critiques
# =====
print('')
print('=' * 75)
print(' AJUSTEMENT DES EXPOSANTS CRITIQUES')
print('=' * 75)

# beta :  $PHI \sim |\tau - \tau_c|^\beta$  cote ordonne (en general  $\tau < \tau_c$ )
# Mais ici a-priori on ne sait pas quel cote, donc on essaie les deux
beta_right, r2_beta_right = fit_exponent(tau_sorted, PHI_means, tau_c_measured, s
beta_left, r2_beta_left = fit_exponent(tau_sorted, PHI_means, tau_c_measured, sid

print('')
print(f' Parametre d ordre  $PHI \sim |\tau - \tau_c|^\beta$  :')
if beta_right is not None:
    print(f' cote  $\tau > \tau_c$  :  $\beta = \{abs(beta\_right):.4f\}$ ,  $R^2 = \{r2\_beta\_ri$ 
if beta_left is not None:
    print(f' cote  $\tau < \tau_c$  :  $\beta = \{abs(beta\_left):.4f\}$ ,  $R^2 = \{r2\_beta\_lef$ 

# Selectionner le meilleur fit
if r2_beta_right is not None and r2_beta_left is not None:
    if r2_beta_right > r2_beta_left:
        beta_fit = abs(beta_right)
        beta_r2 = r2_beta_right

```

```

        beta_side = 'right'
    else:
        beta_fit = abs(beta_left)
        beta_r2 = r2_beta_left
        beta_side = 'left'
else:
    beta_fit = abs(beta_right) if beta_right is not None else (abs(beta_left) if
    beta_r2 = r2_beta_right if r2_beta_right is not None else r2_beta_left
    beta_side = 'right' if beta_right is not None else 'left'

# gamma :  $\chi \sim |\tau - \tau_c|^{-\gamma}$ 
gamma_right, r2_gamma_right = fit_exponent(tau_sorted, chi_means, tau_c_measured,
gamma_left, r2_gamma_left = fit_exponent(tau_sorted, chi_means, tau_c_measured, s

print('')
print(f' Susceptibilite  $\chi \sim |\tau - \tau_c|^{-\gamma}$  :')
if gamma_right is not None:
    print(f' cote  $\tau > \tau_c$  :  $\gamma = \{abs(gamma\_right):.4f\}$ ,  $R^2 = \{r2\_gamma$ 
if gamma_left is not None:
    print(f' cote  $\tau < \tau_c$  :  $\gamma = \{abs(gamma\_left):.4f\}$ ,  $R^2 = \{r2\_gamma\_$ 

if r2_gamma_right is not None and r2_gamma_left is not None:
    if r2_gamma_right > r2_gamma_left:
        gamma_fit = abs(gamma_right)
        gamma_r2 = r2_gamma_right
    else:
        gamma_fit = abs(gamma_left)
        gamma_r2 = r2_gamma_left
else:
    gamma_fit = abs(gamma_right) if gamma_right is not None else (abs(gamma_left)
    gamma_r2 = r2_gamma_right if r2_gamma_right is not None else r2_gamma_left

# alpha :  $C \sim |\tau - \tau_c|^{-\alpha}$ 
alpha_right, r2_alpha_right = fit_exponent(tau_sorted, C_means, tau_c_measured, s
alpha_left, r2_alpha_left = fit_exponent(tau_sorted, C_means, tau_c_measured, sid

print('')
print(f' Chaleur specifique  $C \sim |\tau - \tau_c|^{-\alpha}$  :')
if alpha_right is not None:
    print(f' cote  $\tau > \tau_c$  :  $\alpha = \{abs(alpha\_right):.4f\}$ ,  $R^2 = \{r2\_alpha$ 
if alpha_left is not None:
    print(f' cote  $\tau < \tau_c$  :  $\alpha = \{abs(alpha\_left):.4f\}$ ,  $R^2 = \{r2\_alpha\_$ 

# =====
# COMPARAISON CLASSES D UNIVERSALITE
# =====
print('')

```

```

print('=' * 75)
print(' COMPARAISON AVEC CLASSES D UNIVERSALITE')
print('=' * 75)
print('')
print(f' Exposants Blob mesures : beta = {beta_fit:.3f}, gamma = {gamma_fit:.3f}')
print('')
print(f' {"Classe":18s} {"beta":>8s} {"gamma":>8s} {"ecart_b":>8s} {"ecart_g
print(' ' + '-' * 75)

best_match = None
best_score = float('inf')

for class_name, exps in UNIVERSALITY.items():
    err_beta = abs(beta_fit - exps['beta']) if beta_fit is not None else float('i
    err_gamma = abs(gamma_fit - exps['gamma']) if gamma_fit is not None else floa
    score = err_beta**2 + err_gamma**2
    flag = ''
    if score < 0.01:
        flag = ' *** MATCH ***'
    elif score < 0.05:
        flag = ' * proche *'
    print(f' {class_name:18s} {exps["beta"]:>8.3f} {exps["gamma"]:>8.3f} {err
    if score < best_score:
        best_score = score
        best_match = class_name

print('')
print('=' * 75)
print(f' VERDICT : meilleur match = {best_match} (score {best_score:.4f})')
if best_score < 0.01:
    print(' >> MATCH NET avec une classe d universalite physique connue')
    print(' >> La transition Blob est de meme nature que ' + best_match)
elif best_score < 0.05:
    print(' >> Match approche, signal interessant a creuser')
else:
    print(' >> Pas de match clair avec les classes connues')
    print(' >> Le Blob pourrait avoir sa propre classe d universalite')
print('=' * 75)

# =====
# SAUVEGARDE
# =====

report = {
    'config': {
        'N_fixed': N_FIXED,
        'tau_grid': list(tau_grid),
        'n_seeds': N_SEEDS,

```

```

        'n_equil': N_EQUIL,
        'n_measure': N_MEASURE,
        'total_time_s': total_time,
    },
    'measurements_raw': all_measurements,
    'profiles': {
        'tau_sorted': tau_sorted,
        'PHI_means': PHI_means,
        'chi_means': chi_means,
        'C_means': C_means,
        'D_means': D_means,
    },
    'critical_point': {
        'tau_c_chi_peak': tau_c_measured,
        'chi_peak': chi_peak,
        'tau_c_C_peak': tau_c_C,
    },
    'exponents': {
        'beta': beta_fit,
        'beta_r2': beta_r2,
        'beta_side': beta_side,
        'gamma': gamma_fit,
        'gamma_r2': gamma_r2,
    },
    'best_match': best_match,
    'best_score': best_score,
    'universality_classes': UNIVERSALITY,
}

```

```

report_path = OUTPUT_DIR + '/saut36_RAPPORT.json'
with open(report_path, 'w') as f:
    json.dump(report, f, indent=2, default=str)
print('')
print(f' Rapport sauvegarde : {report_path}')
print(f' Taille : {os.path.getsize(report_path)} octets')

```